

Incompatible API Detection with LLM-Powered Static Analytical Reasoning and Dynamic Simulated Reflection

Abstract—Software version updates are frequent and unavoidable as systems evolve to add new features, enhance performance, and address defects. Among these changes, API modifications represent the most prominent and disruptive form of evolution, directly affecting the stability and usability of downstream applications. While additions and removals of APIs are visible, more subtle internal changes—such as parameter adjustments, altered semantics, and modified behaviors—can also introduce significant compatibility issues. Existing approaches for detecting such problems, including static analysis and syntactic differencing techniques, have achieved partial success but remain limited in their ability to capture semantic nuances and generalize across diverse codebases.

To address these challenges, we present a novel LLM-driven method that leverages the reasoning and code-understanding capabilities of large language models to detect API compatibility issues at both syntactic and semantic levels. Our approach integrates structured change analysis with model-based inference to identify incompatibilities that traditional tools overlook. Through extensive experiments on real-world software projects, we demonstrate that our method outperforms existing baselines, particularly in detecting subtle or complex semantic changes. These results highlight the potential of LLMs to significantly advance the state of the art in API evolution analysis and contribute to more reliable software maintenance.

[?]

I. INTRODUCTION

Software version updates are a routine and inevitable part of modern software development. As software systems evolve to incorporate new features, improve performance, or fix defects, updates are released frequently across ecosystems. This continuous evolution is essential for maintaining software quality and relevance; however, it also introduces challenges for developers who rely on stable interfaces and predictable behavior. Understanding how version updates propagate through software systems is therefore key to ensuring long-term maintainability. In July 2024, a faulty update to CrowdStrike’s Falcon endpoint detection software caused millions of Windows systems worldwide to crash and reboot into blue-screen or recovery mode. This defective update — effectively a broken API/behavior in a widely-deployed component — triggered a global IT outage impacting airlines, hospitals, government services, cloud infrastructure, and more.

Among the various types of changes introduced during software evolution, API modifications represent the most noticeable and impactful updates. APIs serve as the boundary of interaction between components, libraries, and applications. When these interfaces change, downstream systems that depend on them must adapt accordingly. As a result, API updates—whether additions, deletions, or behavioral

modifications—often become the primary source of observable change for developers and tools.

Beyond obvious operations such as adding or removing methods, more subtle internal API changes can also significantly affect compatibility. For example, modifications to parameter types, default values, method semantics, or exception behaviors may not be immediately visible but can still break client code or alter expected functionality. Such implicit or semantic changes are particularly difficult for downstream developers to detect, and their resulting incompatibilities may only surface during runtime or testing, often at considerable cost.

Over the years, a variety of techniques have been proposed to detect API compatibility issues, as reflected in prior work such as [?], [?], [?]. These approaches typically rely on static analysis, syntactic differencing, or rule-based models to identify breaking changes. While effective for many classes of incompatibilities, they still face notable limitations. Many tools struggle with semantic reasoning, cannot generalize across heterogeneous codebases, or fail to capture subtle behavioral differences that arise from complex modifications. Consequently, a reliable and generalizable solution remains elusive.

Recent advancements in large language models (LLMs) present new possibilities for addressing this challenge. The strong reasoning, pattern recognition, and code-understanding capabilities of LLMs have been demonstrated in numerous software-evolution-related tasks [?], [?], [?], such as code migration, patch generation, and refactoring analysis. These successes suggest that LLMs are well suited to infer semantic changes, understand API behavior in context, and detect inconsistencies that traditional static tools overlook.

Motivated by these observations, our work introduces a novel method for XXXX. The proposed approach leverages LLM-based reasoning to analyze API changes at both syntactic and semantic levels, enabling a more comprehensive and context-aware detection of compatibility issues. By integrating structured change analysis with model-driven inference, our method aims to bridge the gap between purely syntactic detection mechanisms and deep semantic understanding.

To evaluate the effectiveness of our approach, we conducted extensive experiments using real-world software projects and diverse API evolution scenarios. The results demonstrate that our method effectively identifies a broader range of compatibility issues compared to baseline tools, particularly in cases that involve complex or subtle semantic changes. Quantitative metrics and qualitative analyses both show that LLM-based reasoning significantly enhances detection accuracy and robustness.

In summary, this work makes several key contributions. First, we provide a systematic analysis of the limitations in existing API compatibility detection methods. Second, we introduce a new LLM-driven technique that unifies syntactic differencing with semantic reasoning. Third, we develop an evaluation framework grounded in real-world software evolution data. Finally, our experimental findings offer evidence that LLMs can meaningfully advance the state of the art in API compatibility analysis.

- We
- We
- We

II. BACKGROUND

A. Python Incompatibility Types

We focus on incompatible APIs in Python, a widely used language whose issues are representative of modern software ecosystems. The incompatible APIs involve the interaction between the upstream dependencies and the downstream client usages. Given an API evolves from an older version to a newer version, the *forward incompatibility* occurs when the newer version introduces changes that prevent existing client code from running correctly, and the *backward incompatibility* occurs when code written for the newer version cannot run correctly on the older version. The incompatible APIs can introduce substantial development overhead and may result in production failures or service disruptions.

Specifically, Python incompatibilities can be categorized into *interface-level changes* and *internal-level changes*. Interface-level changes occur when supported interfaces are modified, such as changes to API signatures or removed entirely, often leading to immediate failures in client code. In contrast, internal-level changes give rise to *behavioral incompatibilities* that are hidden within API implementations. In such cases, the API signature may remain unchanged, allowing client code to execute without errors, yet the program may exhibit altered semantics due to modifications in default parameters, output values, or exception-handling behavior.

As an interpreted language, Python exhibits many dynamic characteristics, which hinder the early identification of incompatibilities. In many cases, such incompatibilities manifest as *behavioral incompatibilities* that affect program execution without causing immediate failures. These issues are more concealed and difficult to detect at an early stage, and therefore tend to lead to more severe consequences.

B. Real-world Python Incompatibilities

We select three real-world Python incompatibilities to illustrate their hidden characteristics and the practical consequences they can cause.

Example of Return Change. This incompatibility originates from the dictionary iteration APIs (e.g., `dict.items()`, `dict.keys()`) in Python. In Python versions earlier than 3.7, the iteration order of dictionaries was unspecified and could vary across executions, whereas Python ≥ 3.7 guarantees that dictionaries preserve insertion order (“*Changed in version 3.7:*

Dictionary order is guaranteed to be insertion order” [29], [31]). As a result, client code that implicitly relies on non-deterministic ordering in older versions, or assumes deterministic ordering in newer versions, may exhibit unexpected or incorrect results when executed under the opposite version, despite using the same API and input data.

Example of Exception Change. This incompatibility arises from the generator execution (e.g., `(__iter__())` semantics in Python. Prior to the adoption of PEP 479 [28], raising `StopIteration` error inside a generator would silently terminate the iteration. After PEP 479, the same behavior results in a raised `RuntimeError`. Consequently, client code that relies on the generator API without explicitly handling this runtime error may execute correctly in older Python versions but crash at runtime in newer versions, even though the generator interface and invocation remain unchanged.

Example of Default Parameter Change. This incompatibility originates from the `sum()` and `mean()` APIs in NumPy, where the default value of the `dtype` parameter changed across versions. In older versions of NumPy (≤ 1.19), these functions used the same data type as the input array for intermediate computations (e.g., `int32`) [36], which could result in silent overflow when processing large integer arrays. In newer versions (≥ 1.20), the default parameter was changed to a larger integer type (e.g., `int64`), preventing overflow [32], [27]. Therefore, client code that relies on the previous behavior may produce different numerical results when executed under a newer version, even when the same input and API calls are used.

C. Categories of Incompatibilities

Following the compatibility types presented in Section II-A, we provide a summary of Python incompatibility categories. We base our categorization on the prior studies by Shen et al. [30] and Zhang et al. [44]. Shen et al. identified incompatibilities arising from parameters (CI1-CI5), exceptions (CI6), and return values (CI7). While these studies define seven primary incompatibility types, our labeling process revealed two additional types, which we include in our categorization.

We use Figure 1 as an exemplary code snippet to aid understanding. The figure shows a merge of the code before (`APIb`) and after (`APIa`) the change, where red lines indicate deletions and green lines indicate additions in the later version.

- **Parameter Deletion/Addition (CI1):** The function parameter is either deleted in the `APIb` or added in the `APIa`. As shown in Figure 1, `amount` is required in `APIb` but removed in `APIa`. Calling `APIa` with `(amount=1)` will raise “`TypeError: fun() got an unexpected keyword argument 'amount'`”.
- **Mismatched Keys in KWargs (CI2):** Different keys may exist in the `APIb` and `APIa`. As shown in Figure 1, `APIb` extracts the value associated with the key ‘`key`’ from `**kwargs`, whereas `APIa` retrieves ‘`key_for_check`’. If the client code invokes `APIa` using the original `kwargs`, a `KeyError` for ‘`key_for_check`’ may be raised.
- **Parameter Rename (CI3):** The function parameter from `APIb` is renamed in the `APIa`. As shown in Figure 1, the parameter `n_size` in `APIb` is renamed to `n_dim` in `APIa`.

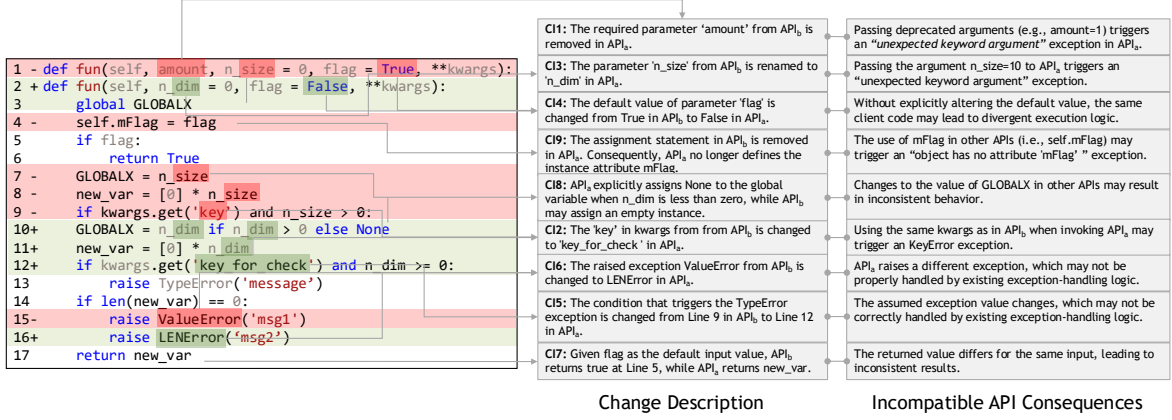


Fig. 1: Illustration of Incompatibility Types

Consequently, invoking API_a with n_size=value raises "TypeError: 'fun()' got an unexpected keyword argument 'n_size'".

- **Parameter Default Value Change (CI4):** The default value of a parameter in API_b is modified in API_a. As shown in Figure 1, the default value of flag changes from True in API_b to False in API_a. Consequently, invoking the API without explicitly specifying flag may lead to a different execution path and produce different results. In this example, API_b may return True at line 6, whereas API_a continues execution to line 17 and returns new_val.
- **Parameter Boundary Value Change (CI5):** The boundary values of the parameter change across API versions, leading to different behaviors when the same parameter value is supplied. As shown in Figure 1, the condition for raising a TypeError changes from n_size < 0 in API_b to n_dim ≤ 0 in API_a. Consequently, providing n_size or n_dim with a value of 0 yields different outcomes. Specifically, an exception is raised in API_a but not in API_b.
- **Exception Change (CI6):** API_b and API_a raise different exceptions under the same conditions, where one API introduces an exception that is not present in the other and may therefore lack appropriate handling logic. As shown in Figure 1, when len(new_val) equals zero, API_b raises a ValueError, whereas API_a raises a LENOError.
- **Return Value Change (CI7):** The returned value changes given the same client invocation. As shown in Figure 1, As noted in CI4, changing a default can cause the API to return different values for the same call (e.g. False in API_{before} vs. new_var(list) in API_a).
- **Global Variable Value Change (CI8):** The value of a global variable may change even when the same API input is used. As shown in Figure 1, both API_b and API_a reference the global variable GLOBALX. When flag is True, API_b assigns GLOBALX = n_size, whereas API_a assigns GLOBALX = n_dim or None.
- **Instance Attribute Uninitialized (CI9):** The instance attribute that exists in one version of an API may become uninitialized in another version. As shown in Figure 1, API_b assigns self.mFlag = flag, whereas API_a does not. Because Python uses a dynamic object model, where instance attributes are created at runtime upon first assignment,

self.mFlag does not exist in API_a.

III. APPROACH

This section details our approach for detecting Python API incompatibilities. We first outline the approach overview, and then elaborate the core components.

A. Challenges and Design Motivation

Existing static analysis methods (e.g., Shen et al. [30]) rely on handcrafted rules. However, rule design requires substantial expert knowledge, making such approaches difficult to generalize to new libraries or previously unseen incompatibilities. Moreover, rule-based methods fail to capture semantic context. For example, when a parameter is renamed from arguments to args, the corresponding AST patterns differ syntactically. As a result, rule-based approaches cannot recognize the underlying rename operation.

Having witnessed the effectiveness of large language models (LLMs) in software engineering tasks^{CITE}, we naturally think of the opportunities of leveraging LLMs to detect incompatible Python APIs. However, recognizing their limitations and to fully harness their potential, we identify the following three challenges in its practical application.

- **I. Linking incompatibility category description to specific code changes.** Each incompatibility category begins with a brief description of how it manifests. However, when such descriptions are provided directly to an LLM, it may fail to fully infer their correspondence to specific code changes. This misalignment between natural-language descriptions and syntactic code changes creates a gap that can hinder the model's effectiveness.
- **II. Scalability with categories of incompatibilities.** Each incompatibility category exhibits distinct syntactic characteristics. A natural approach is to design a dedicated instruction for the LLM for each category. However, as the number of incompatibility categories grows, this approach would require repeated API scanning and reasoning, leading to excessive token consumption and longer execution times, ultimately rendering it impractical for large-scale analyses.
- **III. LLM hallucination and unreliability.** The LLM may generate hallucinated content and exhibit low confidence

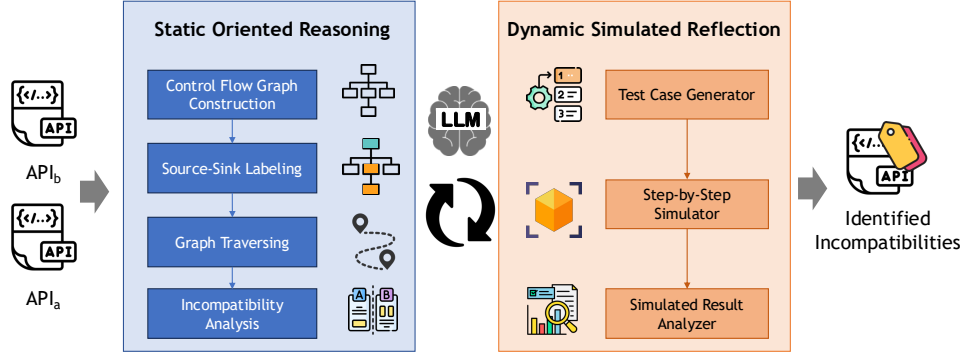


Fig. 2: Overview of COMPACT

in its responses. As a result, some outputs may contain false, contradictory, or inconsistent claims. e.g., citing non-existent code lines to support an assertion.

To address Challenges **I** and **III**, the LLM should fully comprehend the causes of incompatibility from the perspective of code changes and their underlying logic across versions. This requires the LLM to analyze code in a manner similar to a static analyzer, while additionally capturing semantic intent. To tackle Challenge **II**, the LLM should examine the code statements of both versions sequentially from top to bottom and produce intermediate analysis results for each statement. These intermediate states can then be aggregated to determine the final incompatibility category upon completion of the analysis. Furthermore, to mitigate hallucination and unreliability, additional reflection mechanisms are required to validate the LLM’s analysis.

Guided by these requirements, we propose decomposing the reasoning process for incompatibility classification into fine-grained steps. The LLM incrementally computes intermediate states while analyzing each statement and integrates them into a unified, structured reasoning framework, thereby addressing Challenges **I** and **II**. In addition, we incorporate a self-reflection phase in which the LLM validates its static analysis results from the perspective of test-case behavior.

B. Approach Overview

We propose COMPACT. As shown in Figure 2, COMPACT is divided into two phases: *Static-oriented Incompatibility Reasoning* and *Dynamic Simulated Reflection*.

In the first phase, COMPACT performs control-flow graph (CFG) construction, source-sink labeling, graph traversal, and incompatibility analysis. Both API_b and API_a are analyzed once during CFG construction. Subsequently, source-sink labeling and graph traversal are conducted on the constructed CFGs. The notions of source and sink originate from security analysis [CITE](#). In our context, sources correspond to input variables (e.g., parameters), while sinks denote statement endpoints that may trigger incompatibilities. COMPACT then traverses the CFGs to collect statements along paths from sources to sinks. Finally, all incompatibility categories are determined in a single incompatibility analysis step by comparing the source-sink paths and their intermediate data objects. We structure incompatibility reasoning to align LLM understanding with code-level changes, thus addressing Challenge **I**, and design

a workflow that supports multiple incompatibility types, thus addressing Challenge **II**.

In the second phase, COMPACT comprises a test case generator, a step-by-step simulator, and a simulated result analyzer. Unlike the first phase, this phase reflects on the identified incompatibility categories and aims to generate valid test cases that validate the correctness of the determined categories. The process simulates a dynamic execution scenario in which the LLM generates test cases, infers intermediate results as statements are executed, and ultimately confirms the incompatibility category based on these intermediate outcomes. This phase addresses Challenge **III**.

C. Static-oriented Incompatibility Reasoning

This section details our static analysis framework. Unlike traditional compiler-level approaches, we employ a lightweight, semantic abstraction designed to efficiently detect behavioral incompatibilities between API versions.

1) *Control Flow Graph Construction*: The emphasis of our approach is not on classical compiler-level Control Flow Graphs (CFGs), but on a deliberately lightweight, semantic abstraction sufficient for detecting behavioral incompatibilities. At a conceptual level, our CFG is a **condition-indexed execution context graph**, where nodes represent semantic events (assignments, exceptions, returns) and edges are implicit, encoded via condition scope indices rather than explicit jumps. This design implies that explicit control edges, such as jump targets and loops, are unnecessary for CI detection.

The foundation of the CFG is the set of condition statements, denoted as C . Each element in C corresponds to a syntactic condition encountered during the scan of the function body, including `if`, `elif`, `else`, `while`, and exception-handling constructs. Conditions are stored as normalized strings and assigned monotonically increasing indices. These indices serve as stable identifiers for control contexts and function as decision nodes in the implicit CFG. To track the currently active execution context, the construction process maintains a condition stack, CS , while scanning the source code. This stack is updated in a single top-down scan: when the analysis enters a new conditional scope, the corresponding condition index is pushed onto the stack; when the scope ends, it is popped.

Data flow information is captured in the structure D , which represents the primary computational nodes

TABLE I: Categories of API Incompatibilities and Reasoning Process

Id.	Name	Reasoning Process
CI1	Parameter Addition/Deletion	First, the parameters can be extracted into P_{before} and P_{after} from the signature of the function, respectively. Each parameter is in the tuple form of $\langle name, positionindex, type, default \rangle$. Then, CI1 is detected when a parameter (or a positional parameter) appears in P_{before} but not in P_{after} (or vice versa); or an inserted optional parameter in P2 is not last and causes positional shifts.
CI2	Key Deletion of Parameter Kwarg	Extracting the key name access from enumerating the statements of the function bodies into K_{before} and K_{after} . Then, CI2 is detected when K_{before} and K_{after} are not equal.
CI3	Parameter Rename	First, the parameters can be extracted into P_{before} and P_{after} from the signature of the function, respectively. Each parameter is in the tuple form of $\langle name, positionindex, type, default \rangle$. Constructing potential renaming pairs $\langle n_b, n_a \rangle$ by semantic similarity or positional equality. Then, collecting the usage lines of n_b and n_a . i.e., the data flown statements of n_b and n_a , denoted as D_b and D_a , respectively. Finally, CI3 is detected when the D_b and D_a perform same or similar actions.
CI4	Parameter Default Value Change	First, the parameters can be extracted into P_{before} and P_{after} from the signature of the function, respectively. Each parameter is in the tuple form of $\langle name, positionindex, type, default \rangle$. Then, comparing P_{before} and P_{after} default value fields for same-named parameters (or matched by position if appropriate). CI4 is detected where the values are different. Tracing the data flown statements. The data flown statements result in different returned or global values due to the default parameter and causes CI7 and CI8.
CI5	Parameter Boundary Range	First collect the conditions from the beginning to the end and collect the conditions that have data flow relationships to the parameters. For each mapping parameters that both exists in API_{before} and API_{after} , compare the condition boundaries. The CI5 is determined if the parameter boundaries are different.
CI6	Exception Addition/Deletion/Change	Extracting raise statements from API_{before} or API_{after} to R_b and R_a . Record the paths P_{a_b} and P_{a_a} alongside statements from the beginning to the raise statement. Then, compare P_{a_b} and P_{a_a} to see if the raise exceptions are added, deleted or modified. CI6 is detected when at least one of the paths satisfies.
CI7	Return Value Change	Collect the paths alongside from the beginning to the 'return' statement or 'yield' statement. The paths P_{a_b} and P_{a_a} are then compared. The CI7 is determined where a new return expression exists while the other does not contain, or the conditions alongside the path is different that can lead a same API input to come to result in different branches.
CI8	Global Variable Change	Collect the paths alongside from the beginning to the end of the API where the paths contain the global value. The paths P_{a_b} and P_{a_a} are then compared. The CI8 is determined where the conditions alongside the path is different that can lead a same API input to come to result in different branches.
CI9	Reference Parameter Change (e.g., 'self')	In D1/D2, mark entries with variable type = 'reference variable' (including 'self'); collect attribute assignment sites and dependency lists. Statements to detect: assignment statements to reference variables or their attributes, direct reassignments of reference param. Trigger when: the reference parameter itself is reassigned; a new attribute is assigned in one version but not the other; or an existing attribute assignment has materially different assignment logic/dependencies.

of the CFG. Each entry in D corresponds to an assignment statement and records four elements: $(var, var_type, dependencies, condition_indices)$. In CFG terms, each such entry is a statement node whose control dependencies are fully described by its associated condition indices (a snapshot of CS). Explicit control-flow edges are therefore unnecessary, as all relevant control information is encoded in this metadata.

Terminal behaviors of the function are represented by the exception set E and the return set R . Each explicit exception statement—limited strictly to `raise` and `assert` constructs—is recorded in E along with the active condition indices. Similarly, each `return` or `yield` statement is recorded in R with its guarding conditions. Implicit behaviors, such as falling off the end of a function or logging-based warnings, are intentionally excluded because they do not reliably affect API contracts.

The entire CFG is constructed in a single pass over the function body (Algorithm ??). This one-pass design is a critical property. Because the analysis does not require successor resolution, loop unrolling, or interprocedural reasoning, all

necessary control and data dependencies can be captured incrementally. Upon encountering a condition, it is registered and pushed onto the condition stack. Upon encountering an assignment, exception, or return, a node is created with a snapshot of the current stack.

An important aspect of the construction process is its deliberate selectivity. Only entities that are relevant to API-level incompatibility are recorded. Conditions are retained only insofar as they guard observable behaviors. Data flows are recorded only for assignments that may affect return values, exceptions, global state, or reference parameters. Conversely, many elements commonly present in full CFGs are intentionally ignored, including call graphs, logging statements, exception messages, loop iteration counts, and internal refactoring that does not affect externally visible behavior. This pruning is essential for maintaining robustness when the analysis is performed by an LLM.

2) *Source-Sink Labeling*: At a high level, source-sink labeling assigns semantic roles to selected program entities so that data-flow paths relevant to API contracts can be isolated and compared across versions. The key idea is that not all

variables are equally important for API compatibility. Only entities that represent externally observable inputs, externally observable outputs, or persistent side effects are labeled. All other intermediate computations are treated as unlabeled internal glue.

a) Sources: Sources correspond to entities that introduce data into the function from the caller or the external environment. The primary sources are function parameters extracted in Phase 1, including positional parameters, optional parameters with defaults, and `**kwargs`. They define the input surface of the API. Type annotations are explicitly ignored, reinforcing that labeling is driven by runtime semantics. Within parameters, `**kwargs` receives special treatment. The object itself is labeled as a source, but more importantly, individual keys accessed via `kwargs.get('key')` or `kwargs['key']` are labeled as distinct fine-grained sources. This reflects the fact that keys effectively behave as optional named parameters. Keys are only labeled when they are explicitly accessed in the function body; keys that are never read are ignored to prevent over-approximation. Additionally, renamed parameters identified in the *ReName* structure are implicitly treated as corresponding sources across versions. The *ReNU* mapping provides the evidence to align these labels, distinguishing true renames from unrelated changes.

b) Sinks: Sinks represent points where data influences externally observable behavior. The most prominent sinks are **return statements**. Each return expression is labeled as a sink because it directly determines the value observed by the caller. Importantly, return sinks are condition-sensitive: a return expression is associated with the set of active condition indices under which it executes. **Explicit exception statements** form another critical class of sinks. Each `raise` or `assert` statement is labeled as a sink because exceptions alter the control flow visible to callers. The label records the exception type and the guarding conditions, while exception messages are intentionally excluded. Beyond direct outputs, certain **side effects** are labeled as sinks. Assignments to global variables are labeled because they may affect program state beyond the function boundary. Similarly, assignments to reference parameters (e.g., `self` or mutable objects) are labeled when they modify the reference itself or its attributes.

The labeling process is tightly integrated with the one-pass scan used for CFG construction. When a labeled source is introduced, it is recorded once and referenced symbolically. When a labeled sink is encountered, it is recorded together with the current condition stack. Data-flow entries in *D* act as the connective tissue between sources and sinks. Local variables that neither originate from labeled sources nor contribute to labeled sinks are left unlabeled, reducing noise for LLM-based reasoning.

3) Graph Traversing: The graphs produced in earlier phases are implicit rather than explicitly materialized. Control flow is encoded via condition indices, and data flow is encoded through dependency lists in the *D* structure. As a result, graph traversal does not involve explicit edge-following but is realized as structured queries over node sets, guided by labels, dependencies, and control conditions.

Traversal always begins from semantically meaningful

anchor points rather than arbitrary nodes. These anchors include labeled sources (parameters, kwargs keys) and labeled sinks (returns, exceptions, side effects). By restricting traversal origins to these anchors, the analysis avoids unnecessary exploration of internal implementation details. Control sensitivity during traversal is achieved through condition indices rather than explicit branching. When traversing from or to a node, its associated list of condition indices is treated as a path predicate. Two nodes are considered control-aligned if their condition index sets are equivalent or comparable. This allows traversal to reason about execution contexts without constructing explicit true or false edges.

Data-flow traversal is primarily backward and dependency-driven. Starting from a sink node, such as a return statement, traversal proceeds backward through the *D* structure by following dependency relationships. If a return expression depends on variable *v*, traversal identifies assignments to *v* and recursively inspects the variables on which those assignments depend. Traversal is deliberately bounded. It does not compute full transitive closures but stops as soon as one of three conditions is met: a labeled source is reached, a dependency chain diverges structurally between versions, or the remaining dependencies are deemed semantically irrelevant.

Traversal operates in a comparative mode for version alignment. Traversal over the graph of *code_v1* is paired with traversal over *code_v2*. Nodes are aligned based on labels, variable roles, and control contexts. Differences in control flow that do not guard labeled sinks (e.g., refactoring of internal loops) are ignored to maintain a high signal-to-noise ratio.

4) Incompatibility Analysis: The analysis phase assumes that control-flow and data-flow information has already been selectively traversed and aligned across versions. It reasons over the extracted information to determine whether specific compatibility rules are violated.

a) Signature-Level Incompatibilities: These are resolved entirely outside the graph to prevent spurious interactions. This includes **parameter addition/deletion (CI1)** and **default value changes (CI4)**, which are detected through direct comparison of parameter lists.

b) Body-Level Incompatibilities: Using the traversed graph information, we apply deterministic rules to detect the following:

- **Kwargs Incompatibility (CI2):** The analysis compares the sets of explicitly accessed kwargs keys (*K1* and *K2*) identified during traversal. A mismatch indicates a change in the effective input surface. Since unused keys are filtered out, this directly reflects behavioral differences.
- **Parameter Rename Incompatibility (CI3):** We evaluate whether pairs of renamed parameters (from *ReName*) have structurally equivalent data-flow subgraphs. If the extracted subgraphs differ only in variable names but preserve the roles in computations and conditions, it is a valid rename; otherwise, it is an incompatibility.
- **Range Incompatibility (CI5):** The analysis inspects whether parameter-controlled conditions lead to different exception behaviors across versions. This relies on comparing exception nodes together with the identified guarding conditions.

- **Exception Incompatibility (CI6):** This covers differences in exception types, the addition/removal of exception statements, or exception behavior under non-input-related conditions. Separating CI5 and CI6 ensures that range changes and general type changes are not conflated.
- **Return Value Incompatibility (CI7):** We compare aligned return nodes and their data-flow slices. If a return expression in one version has no counterpart in the other, or if upstream data flows produce different values, an incompatibility is reported.
- **Global Variable Incompatibility (CI8):** Differences in the presence of global-variable assignments or their dependency structures indicate potential changes to externally visible shared state.
- **Reference Parameter Incompatibility (CI9):** The analysis inspects assignments to reference parameters and their attributes, focusing strictly on state mutations rather than access patterns.

Across all types, the analysis applies these rules deterministically to the traversed graph information, ensuring that each reported incompatibility is accompanied by a concise, structurally grounded reason.

D. Dynamic Simulated Reflection

To address judgment biases in the incompatibility category candidates generated by the CoT-LLM, we design a multi-agent collaborative validation framework. It implements accurate verification of candidate categories through a process of “test case generation, simulation execution, and comprehensive evaluation,” ultimately filtering out truly valid API incompatibility types. The framework comprises three functionally independent yet collaboratively working agent modules: the Test Case Generation Agent, Simulation Agent, and Comprehensive Analysis Agent. These modules form a validation chain through explicit information interaction, ensuring the objectivity and reliability of validation results.

1) *Test Case Generation:* The Test Case Generation Agent serves as the initiator of the validation process, tasked with constructing targeted test inputs to probe potential incompatibilities. Its core functionality is to generate pairs of calling statements—one for the old API version and one for the new—along with supplementary context, based on the provided old and new API code snippets, the candidate incompatibility category, and the underlying reasoning. The design of these test cases adheres to a critical principle: they must be highly likely to trigger the alleged incompatibility if it indeed exists. For instance, when evaluating a candidate issue related to changed default parameter values, the agent will craft calling statements that omit the affected parameters, thereby exposing discrepancies in behavior caused by the default value shift. The output of this agent follows a structured format, explicitly separating the calling statements and supplementary information for the old and new APIs to ensure clarity in subsequent processing.

The collaborative workflow of the three agents forms a closed-loop validation pipeline. Starting with a candidate incompatibility category, the Test Case Generation Agent

tailors probes to expose potential issues, the Simulation Agent generates empirical behavioral data, and the Comprehensive Analysis Agent renders a data-driven judgment. This division of labor ensures that each stage of validation is handled by a specialized component, reducing the risk of subjective biases that might arise from a single model’s judgment. By structuring the validation as an explicit information flow—from test design to execution to analysis—the framework enhances the transparency and reproducibility of the verification process. Ultimately, this multi-agent approach strengthens the reliability of incompatibility category identification, providing a robust mechanism to filter out spurious candidates and retain only those that are empirically validated.

2) *Step-by-step Simulation:* Upon receiving the test cases from the Test Case Generation Agent, the Simulation Agent takes on the role of executing these tests in a controlled environment. Its primary responsibility is to simulate the runtime behavior of both the old and new API versions when invoked with the generated test inputs. This involves parsing the API code, instantiating necessary objects (if applicable), and executing the calling statements step-by-step. The agent meticulously records the execution results: if the program exits normally with a return value, it captures the specific output; if an exception is raised (e.g., missing arguments, type errors), it documents the error type and description. By faithfully replicating runtime scenarios, this agent transforms abstract code differences into concrete behavioral outcomes, providing empirical data for validating the alleged incompatibility.

3) *Simulated Result Analysis:* The Comprehensive Analysis Agent acts as the final evaluator, synthesizing information from the preceding stages to assess the validity of the candidate incompatibility category. It takes as input the definition of the incompatibility type, the reasoning behind the initial diagnosis, and the behavioral results generated by the Simulation Agent. Using a scoring mechanism ranging from 0 to 10, the agent evaluates the alignment between the diagnosed incompatibility, the test case design, and the observed runtime behaviors. The score is determined based on three key criteria: logical consistency (whether the reasoning logically leads to the alleged incompatibility), rigor of evidence (whether the simulation results clearly demonstrate the expected behavioral discrepancy), and coherence (whether all components of the validation process reinforce the same conclusion). A higher score indicates greater confidence in the existence of the specific incompatibility, while a lower score suggests either flaws in the initial diagnosis or inadequacies in the test scenario.

IV. EVALUATION

A. Dataset Construction

The Python third-party library Application Programming Interface (API) compatibility issue dataset constructed in this study comprises two core components: a benchmark dataset derived from existing research, which includes 108 incompatible API pairs built by Shen et al. on the Flask and pandas libraries; and an extended dataset supplementary constructed in this work. Through systematic repository screening, Issue/Pull Request (PR) analysis, and multi-round validation, the extended dataset

adds over 70 valid samples, resulting in a total of more than 130 compatibility issue annotations. The dataset comprehensively covers third-party libraries across multiple application domains, such as web development, data analysis, and cloud services, and includes API compatibility issues across major, minor, and patch version transitions. This design provides multi-scenario and fine-grained experimental data support for research on Python third-party library API compatibility detection.

Dataset A.

To construct the dataset of 108 incompatible API pairs, Shen et al. selected six version pairs from two representative Python third-party libraries (Flask for web development and pandas for data analysis), covering cross-major, cross-minor, and cross-patch version transitions. A combined approach of regression testing and version update log analysis was adopted to ensure the comprehensiveness and reliability of the dataset. For regression testing, test cases from the previous version (v1) of each library were executed on the updated version (v2); failed test cases indicated potential incompatible APIs, leading to the identification of 70 incompatible API pairs from 58 failed test files among 2,290 regression test files across the six version pairs. Complementarily, version update logs (including intermediate logs for cross-patch version pairs to capture persistent changes) were analyzed to identify additional incompatible APIs by targeting keywords such as "parameter reduction", "added processing", and "no longer supported", resulting in 38 supplementary incompatible API pairs from 11 analyzed logs. To enhance data credibility, the collection process was independently conducted by the first two authors, with discrepancies resolved through group discussions organized by the third author, involving joint reviews of source code, regression test results, and update logs to confirm the existence of compatibility issues for controversial API pairs. This dual-method strategy effectively mitigated limitations associated with incomplete test coverage in unit testing and inconsistent log quality, ultimately forming a dataset of 108 incompatible API pairs that served as the foundation for subsequent empirical analysis.

Dataset B.

We adopted a different strategy to construct our own dataset to cover codes from more areas. We first selected the top 10 Python libraries by download volume over the past 30 days (as of May 1, 2025) from "https://hugovk.github.io/top-pypi-packages/top-pypi-packages-30-days.json". Those libraries are the most popular ones and can represent most python code usage. However, to cover most compatibility issues, for each candidate repository, we also selected additional relevant Python repositories because some issues are not directly reported in the source repository but in relevant Python repositories.

In the preliminary screening of Issues and PRs, for each selected repository, we cycled through a list of keywords (including 'compatibility', 'compatible', 'version', 'evolution', 'exception', 'TypeError', 'AttributeError', 'ImportError', 'breaking changes') and used the GitHub Issues Search API (/search/issues) to retrieve issues and PRs matching these keywords.

Thereafter, approximately 400 entries were filtered through

two rounds of LLM-based checks, followed by manual review. In the LLM analysis phase, the LLM was employed to extract structured information from the issues, including username, repository, version tag1, version tag2, file path, object name, and llm analyse compatibility. Multiple GitHub checks were conducted on the extracted results to ensure the validity of the fields. Specifically, the first LLM was used to convert raw issues into structured compatibility problem entries, which constituted the core information extraction of the dataset, extracting key information such as error type, root cause, library, version, affected files, breaking change, and recommended version. Additionally, the second LLMs was used to supplement and explain the technical details of compatibility issues, especially the causal chain between API changes and errors, serving as an in-depth evidence supplement of the dataset.

After filtering of LLMs, from the original 2000 samples, xxx was ...

Finally, manual verification was performed to confirm the existence of incompatibilities and the correctness of Python API paths, followed by annotation. Ultimately, over 70 samples were obtained, with a total of more than 130 annotations added.

TABLE II: Top Python Libraries by Download Volume

Library	Category	relevant project counts
boto3	Web development	9
urllib3	Web development	7
requests	Web development	10
botocore	Web development	6
certifi	Security	3
charset	xxx	0
normalizer	xxx	0
idna	xxx	3
setuptools	xxx	10
aiobotocore	xxx	3
python-dateutil	xxx	2

B. Experiment Setup

- **Accuracy Evaluation (RQ1):** How is the overall accuracy of COMPACT compared with the state-of-the-arts?
- **Effectiveness Evaluation (RQ2):** How is the effectiveness of COMPACT in different API compatibility types?
- **LLM Generalizability (RQ3):** How is the accuracy of COMPACT using different LLMs?
- **Ablation Study (RQ4):**

Baselines. We selected XXX , XXX, and XXX as the comparison.

Metrics. We use precision and recall as metrics, which are defined as follows:

C. Results

- 1) *Accuracy Evaluation (RQ1):*
- 2) *RQ2:*
- 3) *RQ3:*
- 4) *RQ4:*

V. RELATED WORK

A. API Incompatibility Detection

The API compatibility comes with the subsequence of API evolution, which has been extensively studied. Lamothe et

TABLE III: Evaluation Result Compared with SOTA

Method	Publication	F1	Precision	Accuracy	Recall
Baseline Model A	Shen et al.				
X					
X					
X					
Ours					

TABLE IV: Evaluation Result Compared in Different Compatibility Type

No.	Incompatibility Category	Incom. API (#)	F1	Pre.	Acc.	Rec.
CI1	Param Add or Delete	46				
CI2	Key of Keyword Argument Add or Delete	1				
CI3	Param Rename	7				
CI4	Param Default Value Change	18				
CI5	Param Range Change	59				
CI6	Throw Different Exception	41				
CI7	Return Value Change	94				
CI8	Global Variable Change	3				
CI9	Reference Parameter Change	35				
Macro-Average						

al. [15] conducted in their systematic API evolution review that automatic identifying API change causes and uniform benchmarks as the main remaining challenges. In the context of API compatibility, Cai et al.[3] conducted a large-scale longitudinal study on Android runtime compatibility, while Liu et al.[20] summarized five categories of API-induced compatibility issues. More recently, the focus has deepened into device-specific compatibility behaviors [7] and compatibility issues caused by vendor customizations [4]. Beyond mobile, such challenges are also prevalent in system software like Linux [34], programming languages and its client applications like Java [11] and NPM ecosystem [13]. In deep learning systems, compatibility issues spanning software and hardware stacks have been rigorously characterized [38].

To tackle the problem of incompatibility detection, a natural approach is to construct and execute test cases to trigger API behaviors across different versions. Sun et al. [33] proposed JUnitTestGen, which mines real-world API usages to generate executable unit tests. Going a step further to ensure comprehensive coverage, Mahmud et al. [23] introduced APICIA, which performs a bidirectional impact analysis. Besides, to address client-specific upgrades, Zhu et al. [46] utilized a knowledge-guided framework to generate incompatibility-revealing tests, while Chen et al. [5] leveraged cross-project testing to detect behavioral backward incompatibilities in Python. Beyond

TABLE V: Evaluation Result for Different LLMs

LLMs	F1	Precision	Accuracy	Recall
Deepseek-V3				
X				
X				
X				
X				

TABLE VI: Evaluation Result for Different Method

Evaluation	binary(COT)	ours(one-pass-COT)	ours(one-pass-COT+agent)	ours(one-pass-COT+agent+example)
All-Precision				
All-Recall				
CI1-P				
CI2-P				
CI3-P				
CI4-P				
CI5-P				
CI6-P				
CI7-P				
CI9-P				

standard testing, more sophisticated dynamic techniques have been introduced to analyze complex dependency chains. Kong et al. [14] employed dynamic object relation graphs to precisely detect breaking changes in JavaScript. Furthermore, Cheng et al. propose ReadPyE [6], which addresses the challenge that dynamic analysis requires valid execution environments by automatically generating compatible Python runtimes.

However, testing-based analysis is often limited by high computational costs, its reliance on existing test suites, and inherent coverage issues. In contrast, static analysis offers a more scalable and practical alternative. Addressing device fragmentation, Wei et al. [39] pioneered FicFinder to characterize device-specific issues, a direction later advanced by Pivot [40] to learn API-device correlations from large corpora. To tackle general API evolution, Li et al. [18] introduced CiD to model API lifecycles. He et al. [9] developed IctApiFinder to detect incompatible usages. Beyond general compatibility issues, existing work has examined several specific types of problems, including complex callbacks [24], [10], field-level evolutions [25], AndroMevo-related changes [21], and incompatibilities caused by recurring design smells [12]. In Java, APIDiff [2] and Roseau [16] focus on identifying breaking changes. In Python, static analysis has been tailored to its specific ecosystem. Zhang et al.[44] developed PyCompat to handle framework API evolution, while Zhao et al. [45] constructed knowledge bases to detect version incompatibilities in deep learning stacks. Other studies target specific risks such as breaking changes in packages [8], deprecated APIs [35], and complex variadic parameters [42]. More recently, static approaches have expanded to classify Python incompatibility types [30] and solve cross-repository issues via offline database construction [41].

Despite the advancements, the static-based approaches encounter severe bottlenecks. State what bottlenecks?. To bridge these gaps, we integrate LLMs, enhanced via Chain-of-Thought (CoT) reasoning and multi-agent collaboration to achieve accurate XX.

B. LLMs for Library Evolution

With the rapid advancement of Large Language Models (LLMs), researchers have actively explored their potential in automating complex tasks within the domain of library evolution.

For deprecated APIs, Mahmud et al. [26] proposed GUPPY, the first LLM-based approach to automatically update Android deprecated API usages without requiring manual templates. Similarly, to prevent the introduction of outdated APIs during generation, Bai et al. [1] developed APILOT, which leverages real-time differential analysis to verify LLM outputs against a database of outdated APIs. Zhang et al. [43] enhanced Chain-of-Thought (CoT) by constructing task-specific reasoning patterns. In code synthesis, Li et al. [17] further advanced this by proposing Structured CoT (SCoT), which explicitly embeds programming logic (e.g., branches and loops) into the reasoning chain to constrain the model's output. Liu et al. [22] proposed ERA-CoT to capture entity relationships for better context understanding. Meanwhile, agent-based systems provide another promising approach for library evolution issues. Despite these advancements, fundamental challenges persist in applying LLMs to library evolution. Wang et al. [37] quantitatively revealed that LLMs frequently utilize deprecated APIs due to outdated training data. Liang et al. [19] confirmed that in the Retrieval-Augmented Generation (RAG), the knowledge cutoff significantly hampers performance.

Critically, existing solutions has a drawback that: current CoT approaches and multi-agent frameworks lack design for API compatibility issues. To address these limitations, our work introduces a specialized multi-agent framework. We design a CoT prompt mechanism to explicitly reason about incompatibility categories and orchestrate Input-Generator and Simulator agents to construct a closed-loop verification system.

C. LLM-powered Static Analysis

XXX

LLMDFA: Analyzing Dataflow in Code with Large Language Models

Large Language Model (LLM) for Software Security: Code Analysis, Malware Analysis, Reverse Engineering

Llm-powered static binary taint analysis

VI. CONCLUSION

REFERENCES

- [1] W. Bai, K. Xuan, P. Huang, Q. Wu, J. Wen, J. Wu, and K. Lu, "A pilot: Navigating large language models to generate secure code by sidestepping outdated api pitfalls," *arXiv preprint arXiv:2409.16526*, 2024.
- [2] A. Brito, L. Xavier, A. Hora, and M. T. Valente, "Apidiff: Detecting api breaking changes," in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2018, pp. 507–511.
- [3] H. Cai, Z. Zhang, L. Li, and X. Fu, "A large-scale study of application incompatibilities in android," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 216–227.
- [4] J. Chen, K. Li, Y. Chen, L. Wei, and Y. Liu, "Demystifying device-specific compatibility issues in android apps," in *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2024, pp. 525–537.
- [5] L. Chen, F. Hassan, X. Wang, and L. Zhang, "Taming behavioral backward incompatibilities via cross-project testing and analysis," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 112–124.
- [6] W. Cheng, W. Hu, and X. Ma, "Revisiting knowledge-based inference of python runtime environments: A realistic and adaptive approach," *IEEE Transactions on Software Engineering*, vol. 50, no. 2, pp. 258–279, 2023.
- [7] Z. Dong, Y. Zhao, T. Liu, C. Wang, G. Xu, G. Xu, L. Zhang, and H. Wang, "Same app, different behaviors: Uncovering device-specific behaviors in android apps," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2099–2109.
- [8] X. Du and J. Ma, "Aexpy: Detecting api breaking changes in python packages," in *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2022, pp. 470–481.
- [9] D. He, L. Li, L. Wang, H. Zheng, G. Li, and J. Xue, "Understanding and detecting evolution-induced compatibility issues in android apps," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 167–177.
- [10] H. Huang, L. Wei, Y. Liu, and S.-C. Cheung, "Understanding and detecting callback compatibility issues for android applications," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 532–542.
- [11] D. Jayasuriya, V. Terragni, J. Dietrich, and K. Blincoe, "Understanding the impact of apis behavioral breaking changes on client applications," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1238–1261, 2024.
- [12] W. Jin, J. Shang, J. Zheng, M. Sun, Z. Huang, M. Fan, and T. Liu, "The design smells breaking the boundary between android variants and aosp," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2025, pp. 599–599.
- [13] D. Kong, J. Liu, L. Bao, and D. Lo, "Toward better comprehension of breaking changes in the npm ecosystem," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, pp. 1–23, 2025.
- [14] D. Kong, J. Liu, C. Ni, D. Lo, and L. Bao, "More effective javascript breaking change detection via dynamic object relation graph," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 2340–2361, 2025.
- [15] M. Lamothe, Y.-G. Guéhéneuc, and W. Shang, "A systematic review of api evolution literature," *ACM Computing Surveys (CSUR)*, vol. 54, no. 8, pp. 1–36, 2021.
- [16] C. Latappy, T. Degueule, J.-R. Falleri, R. Robbes, and L. Ochoa, "Roseau: Fast, accurate, source-based api breaking change analysis in java," *arXiv preprint arXiv:2507.17369*, 2025.
- [17] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–23, 2025.
- [18] L. Li, T. F. Bissyandé, H. Wang, and J. Klein, "Cid: Automating the detection of api-related compatibility issues in android apps," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2018, pp. 153–163.
- [19] L. Liang, J. Gong, M. Liu, C. Wang, G. Ou, Y. Wang, X. Peng, and Z. Zheng, "Rustev0 2: An evolving benchmark for api evolution in llm-based rust code generation," *arXiv preprint arXiv:2503.16922*, 2025.
- [20] P. Liu, Y. Zhao, H. Cai, M. Fazzini, J. Grundy, and L. Li, "Automatically detecting api-induced compatibility issues in android apps: a comparative analysis (replicability study)," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 617–628.
- [21] P. Liu, Y. Zhao, M. Fazzini, H. Cai, J. Grundy, and L. Li, "Automatically detecting incompatible android apis," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 1, pp. 1–33, 2023.
- [22] Y. Liu, X. Peng, T. Du, J. Yin, W. Liu, and X. Zhang, "Era-cot: improving chain-of-thought through entity relationship analysis," *arXiv preprint arXiv:2403.06932*, 2024.
- [23] T. Mahmud, M. Che, J. Rouijel, M. Khan, and G. Yang, "Apicia: An api change impact analyzer for android apps," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 99–103.
- [24] T. Mahmud, M. Che, and G. Yang, "Acid: an api compatibility issue detector for android apps," in *Proceedings of the ACM/IEEE 44th international conference on software engineering: Companion proceedings*, 2022, pp. 1–5.
- [25] T. Mahmud, M. Che, and G. Yang, "An empirical study on compatibility issues in android api field evolution," *Information and Software Technology*, vol. 175, p. 107530, 2024.
- [26] T. Mahmud, B. Duan, M. Che, A. Yasmin, A. H. Ngu, and G. Yang, "Automated update of android deprecated api usages with large language models," *arXiv preprint arXiv:2411.04387*, 2024.
- [27] numpy.org, "Numpy reference," 2025. [Online]. Available: <https://numpy.org/doc/1.20/numpy-ref.pdf>
- [28] python.org, "Pep 479 – change stopiteration handling inside generators," 2025. [Online]. Available: <https://peps.python.org/pep-0479/>

- [29] readthedocs.io, “Python 3.12.0a0 documentation,” 2025. [Online]. Available: <https://pradyunsg-cpython-lutra-testing.readthedocs.io/en/latest/library/stdtypes.html>
- [30] K. Shen, K. Huang, B. Chen, and X. Peng, “Detecting incompatible third-party library apis in python based on static analysis,” *Journal of Software (Chinese)*, vol. 36, no. 4, pp. 1435–1460, 2024.
- [31] stackoverflow.com, “Is the order of a python dictionary guaranteed over iterations?” 2025. [Online]. Available: <https://stackoverflow.com/questions/2053021/is-the-order-of-a-python-dictionary-guaranteed-over-iterations>
- [32] stackoverflow.com, “Numpy function type casting automatically or not,” 2025. [Online]. Available: <https://stackoverflow.com/questions/70079170/numpy-function-type-casting-automatically-or-not>
- [33] X. Sun, X. Chen, Y. Zhao, P. Liu, J. Grundy, and L. Li, “Mining android api usage to generate unit test cases for pinpointing compatibility issues,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.
- [34] C.-C. Tsai, B. Jain, N. A. Abdul, and D. E. Porter, “A study of modern linux api usage and compatibility: What to support when you’re supporting,” in *Proceedings of the Eleventh European Conference on Computer Systems*, 2016, pp. 1–16.
- [35] A. Vadlamani, R. Kalicheti, and S. Chimalakonda, “Apiscanner-towards automated detection of deprecated apis in python libraries,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2021, pp. 5–8.
- [36] w3cub.com, “numpy.sum,” 2025. [Online]. Available: <https://docs.w3cub.com/numpy/1.17/generated/numpy.sum>
- [37] C. Wang, K. Huang, J. Zhang, Y. Feng, L. Zhang, Y. Liu, and X. Peng, “Llms meet library evolution: Evaluating deprecated api usage in llm-based code completion,” *arXiv preprint arXiv:2406.09834*, 2024.
- [38] J. Wang, G. Xiao, S. Zhang, H. Lei, Y. Liu, and Y. Sui, “Compatibility issues in deep learning systems: Problems and opportunities,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 476–488.
- [39] L. Wei, Y. Liu, and S.-C. Cheung, “Taming android fragmentation: Characterizing and detecting compatibility issues for android apps,” in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 226–237.
- [40] L. Wei, Y. Liu, and S.-C. Cheung, “Pivot: learning api-device correlations to facilitate android compatibility issue detection,” in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 878–888.
- [41] Y. Xie, Z. Jia, S. Li, Y. Wang, J. Ma, X. Li, H. Liu, Y. Fu, and X. Liao, “How to pet a two-headed snake? solving cross-repository compatibility issues with hera,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 694–705.
- [42] S. Zhang, G. He, and G. Xiao, “Coding pitfalls: Demystifying the potential api compatibility risk of variadic parameters in python,” in *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2024, pp. 105–106.
- [43] Y. Zhang, X. Wang, L. Wu, and J. Wang, “Enhancing chain of thought prompting in large language models via reasoning patterns,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25 985–25 993.
- [44] Z. Zhang, H. Zhu, M. Wen, Y. Tao, Y. Liu, and Y. Xiong, “How do python framework apis evolve? an exploratory study,” in *2020 IEEE 27th international conference on software analysis, evolution and reengineering (saner)*. IEEE, 2020, pp. 81–92.
- [45] Z. Zhao, B. Kou, M. Y. Ibrahim, M. Chen, and T. Zhang, “Knowledge-based version incompatibility detection for deep learning,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 708–719.
- [46] C. Zhu, M. Zhang, X. Wu, X. Xu, and Y. Li, “Client-specific upgrade compatibility checking via knowledge-guided discovery,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–31, 2023.